

Hixean: cómo programar en él

Un compilador AOT a C11, sin máquina virtual

Hixean 0.2.0
octubre de 2026
github.com/wwwillcoxon/Hixean

Contenido

Antes de escribir una línea	3
El primer programa	3
Dónde van los archivos	3
Variables y tipos	5
Control de flujo	6
Funciones	7
Registros	8
Aritmética: verificada, y con intención explícita	9
Operadores propios	10
Bits	11
Cadenas	12
Arreglos	13
MAYBE, NIL, Result y `?`	14
MATCH y patrones	15
Genéricos, restricciones y traits	16
Iteradores	17
Vectores	18
Módulos y paquetes	19

Antes de escribir una línea

Hixean compila a C11 y el C se compila con el compilador del sistema. No hay máquina virtual, ni recolector de basura, ni *unwinding*. Eso tiene una consecuencia práctica que conviene tener presente desde el principio: **lo que el programahace está en el C que ves cuando falla**. Un `hxc build` deja el generado en `build/gen/`, y se puede abrir.

El perfil por defecto es `freestanding`: `syscalls` directas, sin `libc`, memoria de la arena y del sistema. Un hola mundo ocupa 8 896 bytes. El perfil `libc` existe para cuando quieras `stdio` y compañía.

El primer programa

Una extensión `.hxe` es un programa ejecutable; una `.hxf` es una biblioteca. El punto de entrada no se declara: es el archivo.

salida

Para ejecutarlo:

Para verlo en hexadecimal, que es la otra mitad de este lenguaje:

Dónde van los archivos

| sufijo | qué es | |---|---| | `.hxe` | programa ejecutable | | `.hxf` | biblioteca de funciones exportables | | `.hxs` | módulo interno, para `IMPORT` | | `.hxt` | prueba: su salida esperada vive en `.hxt.out` | | `.hxx` | manifiesto de paquete | | `.hxq` | consulta de paquetes | | `.hxc` | biblioteca publicada, interfaz más código compilado |

Una prueba es un programa con su salida esperada al lado. Eso es todo lo que es una prueba en Hixean: no hay un `assert`, hay un fichero de texto.

salida

Variables y tipos

Se declaran con DIM, siempre antes de usarlas. Los tipos incorporados son BOOL, INT, I64, FLOAT, STRING y DURATION, más MAYBE T, ARRAY[T], REF T y PTR T.

Las variables son valores, no referencias: copiar copia. Un TYPE es un registro y también se copia. El verificador es estricto en una cosa que sorprende viniendo de otros lenguajes: **detrás de un `MAYBE` hay que mirar**

```
hay: hola
salida
```

Un T se convierte solo a MAYBE T. Al revés no: un MAYBE STRING donde se espera un STRING es un error, y con razón, porque el valor puede no estar.

Control de flujo

IF con ELSEIF y ELSE. Después de THEN puede venir una línea o un bloque.

```
positivo
salida
```

WHILE y FOR. El FOR tiene TO y STEP, y el cuerpo va hasta NEXT.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

BREAK sale del bucle, CONTINUE pasa a la siguiente vuelta, EXIT sale de la función.

Funciones

```
salida
```

Un parámetro puede tener valor por defecto, y entonces se puede omitir:

```
Hola, ana
Buenos días, ana
```

REF es un préstamo explícito: quien recibe promete devolverlo. Se escribe DIM d AS REF C = k al declarar, y reasignar la variable después es un error (E0408), porque el préstamo ya está en otro sitio.

Registros

Un TYPE es un registro con nombre: sus campos llevan tipo, y se inicializan con DIM. No tiene métodos; los métodos son de los traits, que es lo que se ve más abajo.

salida⁴

Para el comportamiento hay dos caminos. Uno es el operador propio del que hemos hablado. El otro, cuando lo que quieres es añadir una misma operación a varios tipos:

salida⁵ un punto

El despacho es estático: IMPLEMENTAR se resuelve al compilar, no hay tabla virtual, y el cuerpo de la función llama al método por el trait.

Aritmética: verificada, y con intención explícita

La suma y la resta sobre enteros **comprueban el desbordamiento** y abortan con el código 70. Es una decisión, no un descuido: en un lenguaje de sistemas es mejor un programa que para que saber que el número no cabía.

| operadores | qué hacen | |---|---| | + - | verificados: trampa y salida 70 si desbordan | | * / MOD | sin verificar, como en C | | +% -% | envolvente: dan la vuelta | | + | - | * | | saturante: se quedan en el máximo o el mínimo |

salida 294967296

Con + ese mismo programa no imprimiría nada: 2000000000 + 2000000000 no cabe en un INT de 32 bits, así que aborta con el código 70 y un mensaje en la salida de error. Probalo, es la forma más rápida de entender la decisión.

Si lo que quieres es el resultado del entorno que al dar la vuelta, pídeselo con el operador que lo dice: a +% a. Escribir + y esperar que no compruebe, o escribir +% esperando que pare, son los dos errores que esta separación evita.

La división por cero se comprueba también, y con divisor constante el error es de compilación: es mejor no generar código que aborta.

Operadores propios

Un TYPE puede definir cómo se comporta con los operadores del lenguaje. El mangling es `op_<nombre>__<Tipo>`, así que dos sobrecargas del mismo signo con tipos distintos conviven.

```
5/6
sanda
false
false
```

Se admiten `+` `-` `*` `/` `MOD` `++` `==` `<>` `<` `<=` `>` `>=` y los de aritmética explícita `(+% -% +| -| *|)`.
`!=` es otra forma de `<>`: sobrecargar los dos es un error, porque son el mismo operador.
El tipo del primer parámetro manda. `a * 6` con `OPERATOR *` (`a` AS `Fraccion`, `b` AS `INT`) funciona. Un parámetro que no sea de un TYPE declarado da E0215.

Bits

AND, OR y XOR leen de dos maneras según el tipo de los operandos: con BOOL son los operadores booleanos, y con INT o I64 son la operación de bits. ~ es el complemento a bits, y << y >> mueven posiciones.

```
no se puede ejecutar
salida
puede leer
-12487
```

Mezclar un BOOL con un entero es un error (E0307), con una nota que dice que a un lado le falta el otro. El lenguaje no convierte un BOOL a entero por su cuenta, ni al revés.

Y ojo con la precedencia, que es la de C: == se ata más fuerte que AND, así que `x AND 1 == 1` se lee como `x AND (1 == 1)`. Los paréntesis no son decorativos ahí.

El desplazamiento está comprobado: con cuenta constante lo dice el verificador (E0316) y con cuenta variable lo dice el runtime, que aborta con el mismo código 70 que el desbordamiento de la suma. El C no promete nada en ninguno de los dos casos.

Cadenas

STRING es una estructura con puntero y longitud: los literales se comparan por contenido y no se pueden modificar en su sitio. ++ concatena.

```

HOLA, mundo
4
HOLA
0
```

Los métodos de STRING son Len, IsEmpty, Upper, Lower, Trim, Slice, At y Repeat.
Para poner un número dentro de un texto está ToString(), que existe en INT, I64, FLOAT, BOOL y DURATION:

```

salida 42
```

El texto que devuelve vive en memoria propia y **no se libera** Hixean no tiene recolector y no va a fingir uno. Quien llame muchas veces en un bucle debe encerrarlo en un ARENA.
La biblioteca estándar de texto es un módulo más:

```

true
true
007
```

Arreglos

Un arreglo tiene su tamaño en el tipo, y por eso `Len()` es una constante: no cuesta código.

```
5
sáida
9
9
```

`a[i]` es rápido y **no compruebanada**, como en casi todos los lenguajes. `a.At(i)` comprueba el índice y aborta con un número si no cabe. Quien quiera el diagnóstico usa `At`.

El arreglo dinámico se declara `ARRAY[T]` o `T[]`, y crece con `Push`:

```
5
sáida
40
99
```

Cuando crece no copia el arreglo entero con `realloc`: reserva un bloque nuevo y copia. `realloc` no existe en el perfil `freestanding`, y copiar a mano es predecible. La memoria sale de una arena, así que un `Push` dentro de un bucle que se repite mucho crece la arena; para eso está `ARENA`.

MAYBE, NIL, Result y?

MAYBE `T` es un valor de tipo `T` o nada. `NIL` solo vale dentro de un `MAYBE`: fuera de él es un error (E0211), no un cero disfrazado. El acierto se ve con `IsNil()`, se sustituye con `Or(x)` y se transforma con `Map(f)`.

`Result<T,E>` es lo que casi todo lenguaje llama `Either` o `Result`, con dos parámetros: el del éxito y el del error, que aquí es `STRING`.

El `?` toma el valor del `Ok` o propaga el error: es el camino corto para no escribir el mismo `CASE Err` en cada función. Fíjate en `Result<INT, STRING>`: un `Result` sin los dos parámetros no sabe qué lleva dentro, y tanto `?` como `CASE Ok(v)` lo dicen con E0219 en vez de adivinar.

error de division por cero caracteres

En cualquier otro tipo, el `MATCH` necesita `CASE ELSE` (E0405).

MATCH y patrones

```
baio
salida
alto
```

El guard ve lo que el patrón liga, así que `CASE n WHEN n > 10` usa la `n` que ese mismo `CASE` acaba de ligar. Un patrón puede traer un rango (`CASE 1 TO 5`), y | varias alternativas. Un `CASE` con dos alternativas que ligan el mismo nombre es E0217: el cuerpo no podría saber de cuál de las dos habla. Un `CASE` por alternativa se entiende mejor.

Genéricos, restricciones y traits

Un TYPE o una FUNCTION genérica se declara con `<T>`, y DONDE exige un trait. La comprobación ocurre al instanciar, en el punto de la llamada, no dentro del cuerpo.

No hay herencia y hay subtipado estructural: dos TYPE con la misma forma son compatibles donde se pida el más pequeño, sin declarar una interfaz por cada interfaz.

Los traits se despachan de forma estática: no hay tabla virtual ni coste en tiempo de ejecución, porque la resolución es al compilar.

Iteradores

FOR ... IN con un iterador perezoso. Los verbos son pocos a propósito: Rango, RangoF, Map, Filter, Take y First.

salida

4
9
2
4
6

Una FUNC se eleva a una función del módulo y se referencia como puntero a función. **No captura el entorno** una lambda que intente leer una variable de su alrededor da un error de verificación, no un fallo raro. Es la decisión documentada en ADR 0008, y la que hay que revisar si algún día hace falta.

Vectores

VEC2, VEC3 y VEC4 son FLOAT agrupados: tres FLOAT en la pila, sin puntero ni indirección. Se escriben con un literal y se imprimen como sus componentes.

Cada verbo va en dos formas, la de método y la de función: `a.DOT(b)` es `DOT(a, b)`. Un nombre de dos a cuatro letras es un *swizzle*: `a.zy` es un VEC2 con los componentes en el orden escrito. MAT4 y QUAT existen en el runtime pero **no tienen nombre** no hay manera de construirlos, y dar el nombre sin constructor sería una promesa que el lenguaje no puede cumplir.

Módulos y paquetes

Un módulo importable es un `.hxs`. El namespace es el último segmento del nombre: `IMPORT std.texto` se usa como `texto`.